

Dilithium 메모리 최적 구현 코드 분석

유튜브 주소 : <https://youtu.be/vab7CW79TxY>

Dilithium 개요



- Dilithium: NIST 공모전에 제출된 PQC 전자서명 알고리즘
- NIST 공모전: PQC 알고리즘을 평가해 표준을 선정하는 공모전
 - ML-KEM(FIPS203): 키교환/키캡슐화 표준(Kyber-격자기반)
 - **ML-DSA(FIPS204): 전자서명 표준(Dilithium-격자기반)**
 - SLH-DSA(FIPS205): 전자서명 표준(SPHINCS+-해שי기반)
 - 이외 FALCON(격자기반), HQC(코드기반) 알고리즘이 표준화 진행 중
- **Dilithium의 기반 난제: 격자(module-lattice)**
 - 격자기반: 보통 성능/구현 효율이 좋음 -> 대부분 표준 후보가 격자 기반
 - **NTT를 이용해 빠른 다항식 연산이 가능함**
 - 보통 성능 병목이 NTT에서 발생
 - **성능 상 병목은 NTT가 주요 파트이나, NTT의 RAM Peak는 높지 않음**

메모리 최적 구현 분석 대상

- 연구 목적: Dilithium의 RAM peak 감소
- 메모리 점유의 가장 큰 원인은 NTT가 아님
 - 공개 행렬과 다항식 벡터의 곱($A \cdot v$) 상에서 발생
 - 공개 행렬인 A 를 메모리에 보관할 때가 RAM peak
- 메모리 최적화가 필요한 이유?
 - IOT/임베디드 환경에선 RAM이 제한됨
 - 수십KB~수KB 정도로 제한
 - ML-DSA 레퍼런스 구현의 경우 Sign 시 87KB 이상 사용
 - 이 중 행렬 A 만해도 약 30KB 차지(Dilithium3 기준)
- 즉, 타겟 연구는 행렬 A 를 줄이는데만 초점을 맞춤
 - NTT의 최적화는 향후 연구로만 언급
 - ROM 최적화도 수행하지 않음, 오직 RAM Peak 절감이 목표
- Pure C로만 최적 구현을 수행하였기에, 타겟 연구로 적합했음



Compact Memory Implementations of the ML-DSA Post-Quantum Digital Signature Algorithm

Rodrigo Duarte de Meneses, Caio Teixeira, Marco Aurélio Amaral Henriques

¹Faculdade de Engenharia Elétrica e de Computação

Universidade Estadual de Campinas (Unicamp) – Campinas, SP – Brasil

r197962@dac.unicamp.br, caio@dca.fee.unicamp.br, maah@unicamp.br

Abstract. This paper explores memory optimization techniques in the implementation of the Module-Lattice-Based Digital Signature Algorithm (ML-DSA) in the context of post-quantum cryptography. It shows how to achieve significant reductions in memory usage, and evaluates the trade-offs in computational speed. Moreover, it demonstrates how the secret (private) key can be managed to reduce significantly its storage requirements, thereby enhancing ML-DSA's applicability in some resource-constrained environments.

1. Introduction

With the recent advent of quantum computing, the cryptographic research community faces significant challenges due to the potential capabilities of quantum computers. One of the most critical issues is the threat quantum computers pose to current cryptographic systems, particularly public key algorithms like RSA (Rivest-Shamir-Adleman) and ECC (Elliptic Curve Cryptography) [Paar and Pelzl 2010]. The primary concern arises from Shor's algorithm [Shor 1994], a quantum algorithm that can efficiently solve the integer factorization and discrete logarithm problems, which are foundational to RSA and ECC, respectively. This capability threatens the security of vast amounts of data currently protected by these cryptographic schemes, potentially rendering them obsolete.

In response to these emerging challenges, post-quantum cryptography (PQC) has gained prominence. PQC aims to develop cryptographic algorithms secure against both classical and quantum attacks. Recognizing the importance of establishing robust post-quantum standards, the National Institute of Standards and Technology (NIST) initiated an international competition in 2016 to evaluate and select quantum-resistant cryptographic standards [NIST 2016]. This competition has identified several promising candidates, including lattice-based, hash-based, code-based, and isogeny-based cryptosystems. Each category has unique strengths and weaknesses but all share the common goal of resisting quantum computational attacks.

Among the various candidates, the Module-Lattice-Based Digital Signature Algorithm (ML-DSA) was defined as one of the standards for post-quantum digital signatures [NIST 2024]. ML-DSA is based on the CRYSTALS-Dilithium algorithm, a leading lattice-based digital signature scheme that has shown good performance in terms of security and efficiency [Lyubashevsky et al. 2021]. CRYSTALS-Dilithium leverages the hardness of lattice problems, specifically structured lattices, which are believed to be secure against both classical and quantum attacks. Lattice-based cryptography, in general, offers several advantages, including worst-case hardness guarantees and versatility in constructing various cryptographic primitives [Regev 2005].

적용된 메모리 최적화 기법

- **메모리 최적 구현 핵심 전략: on-the-fly(행렬 A 비물질화)**
 - 행렬 A는 $\text{seed}(p)$ 로부터 생성 가능 - SHAKE128을 통해 생성됨
 - 시드만 있으면 같은 행렬을 언제든지 다시 만들 수 있음 $\rightarrow$ RAM에 전체를 저장할 필요 X
 - 행렬 A 전체를 RAM에 저장하지 않고, 필요할 때마다 생성하여 사용 후 버림
- **위 전략을 구현하기 위해 3가지 기법이 사용됨**
 - **기법1: 커널 융합**
 - 행렬을 먼저 생성하고 곱셈을 수행하는게 아닌, 생성과 동시에 곱셈 수행
 - **기법2: 스트리밍 단위 축소**
 - 여러 다항식을 한 번에 생성하는게 아닌, 다항식 1개 단위로 생성 $\rightarrow$ 임시 버퍼 크기 감소
 - 블록 크기가 줄어 RAM peak는 줄어드나, 성능은 느려짐
 - **기법3: 버퍼 재사용**
 - 지역 변수($\text{mat}[K]$) 대신 작은 버퍼(scratch 버퍼 - poly p)를 선언 후 반복하여 재사용
- 메모리 - 성능 간 trade-off 존재(Sign 사이클 약 47% 증가)

기법1: 커널 융합

호출부 변화

```
/*  
polyvec_matrix_expand(mat, rho);  
polyvec_matrix_pointwise_montgomery(&t1, mat, &s1hat);  
*/  
polyvec_matrix_expand_and_multiply(&t1, rho, &p, &s1hat);
```

• 생성(expand)과 곱셈(pointwise)를 분리하던 구조 -> 한 함수로 합침

- expand 직후 pointwise 곱셈 및 누적까지 한 루프에서 처리
-> 중간 저장($K \times L$ 다항식-mat) 제거

• 기존 구현: 2개 함수

- polyvec_matrix_expand
- ...pointwise_montgomery
- **mat[K] 필요**

• 최적 구현: 1개 함수

- ...expand_and_multiply
- **mat[K] 제거**
- **최적구현은 모두 오른쪽 모듈**
- **내에서 3가지 기법 모두 사용된다**
- **고 말할것**

```
void polyvec_matrix_expand_and_multiply(polyveck *t, const uint8_t rho[SEEDBYTES], poly *p, const polyvec1 *v) {  
    unsigned int i, j;  
    poly w;  
  
    for(i = 0; i < K; ++i){  
        /* for(k = 0; k < N; ++k){  
            t->vec[i].coeffs[k] = 0;  
        }  
        Forma alternativa sem a necessidade de utilizar o memset*/  
  
        memset(t->vec[i].coeffs, 0, sizeof(t->vec[i].coeffs));  
        for(j = 0; j < L; ++j){  
            poly_uniform(p, rho, (i << 8) + j);  
            poly_pointwise_montgomery(&w, p, &v->vec[j]);  
            poly_add(&t->vec[i], &t->vec[i], &w);  
        }  
    }  
}  
  
/*void polyvec_matrix_expand(polyvec1 mat[K], const uint8_t rho[SEEDBYTES]) {  
    unsigned int i, j;  
  
    for(i = 0; i < K; ++i)  
        for(j = 0; j < L; ++j)  
            poly_uniform(&mat[i].vec[j], rho, (i << 8) + j);  
}  
  
void polyvec_matrix_pointwise_montgomery(polyveck *t, const polyvec1 mat[K], const polyvec1 *v) {  
    unsigned int i;  
  
    for(i = 0; i < K; ++i)  
        polyvec1_pointwise_acc_montgomery(&t->vec[i], &mat[i], v);  
}*/
```

최적 구현

기존 구현

기법2: 스트리밍 단위 축소

- 여러 다항식을 한 번에 생성하는게 아닌, 다항식 1개 단위로 생성
 - RAM에 올라가는 다항식이 30개에서 1개로 줄어듦
- 기존 구현: 행렬 A 전체 생성
 - $K \times L$ (6*5)개 한 번에 생성 후 유지
- 최적 구현: 1개씩 생성
 - 1개 다항식(p) 생성
 - 이후 바로 곱셈(pointwise)
 - 이후 누산(add)

```
void polyvec_matrix_expand_and_multiply(polyveck *t, const uint8_t rho[SEEDBYTES], poly *p, const polyvecl *v) {
    unsigned int i, j;
    poly w;

    for(i = 0; i < K; ++i){
        /* for(k = 0; k < N; ++k){
            t->vec[i].coeffs[k] = 0;
        } */
        /* Forma alternativa sem a necessidade de utilizar o memset */
        memset(t->vec[i].coeffs, 0, sizeof(t->vec[i].coeffs));
        for(j = 0; j < L; ++j){
            poly_uniform(p, rho, (i << 8) + j);
            poly_pointwise_montgomery(&w, p, &v->vec[j]);
            poly_add(&t->vec[i], &t->vec[i], &w);
        }
    }

    /*void polyvec_matrix_expand(polyvecl mat[K], const uint8_t rho[SEEDBYTES]) {
        unsigned int i, j;

        for(i = 0; i < K; ++i)
            for(j = 0; j < L; ++j)
                poly_uniform(&mat[i].vec[j], rho, (i << 8) + j);
    } */

    void polyvec_matrix_pointwise_montgomery(polyveck *t, const polyvecl mat[K], const polyvecl *v) {
        unsigned int i;

        for(i = 0; i < K; ++i)
            polyvecl_pointwise_acc_montgomery(&t->vec[i], &mat[i], v);
    }
}
```

1개 생성
바로 곱셈
이후 누적

30개 다항식 동시 생성

기법3: 버퍼 재사용

- 지역 변수(mat[K]) 대신 작은 버퍼(scratch 버퍼 - poly p)를 선언
- 이후 p를 반복하여 재사용(덮어쓰며)

기존 구현

```
int PQCLEAN_MLDSA65_CLEAN_crypto_sign_keypair(uint8_t *pk, uint8_t *sk) {
    uint8_t seedbuf[2 * SEEDBYTES + CRHBYTES];
    uint8_t tr[TRBYTES];
    const uint8_t *rho, *rhoprime, *key;
    polyvec1 mat[K];
    polyvec1 s1, s1hat;
    polyveck s2, t1, t0;
```

함수 시작 시 30KB 할당

```
/* Get randomness for rho, rhoprime and key */
randombytes(seedbuf, SEEDBYTES);
seedbuf[SEEDBYTES + 0] = K;
seedbuf[SEEDBYTES + 1] = L;
shake256(seedbuf, 2 * SEEDBYTES + CRHBYTES, seedbuf, SEEDBYTES + 2);
rho = seedbuf;
rhoprime = rho + SEEDBYTES;
key = rhoprime + CRHBYTES;
```

mat은 함수 끝까지 RAM에 상주

```
/* Expand matrix */
PQCLEAN_MLDSA65_CLEAN_polyvec_matrix_expand(mat, rho);
```

```
/* Sample short vectors s1 and s2 */
PQCLEAN_MLDSA65_CLEAN_polyvec1_uniform_eta(&s1, rhoprime, 0);
PQCLEAN_MLDSA65_CLEAN_polyveck_uniform_eta(&s2, rhoprime, L);
```

```
/* Matrix-vector multiplication */
s1hat = s1;
PQCLEAN_MLDSA65_CLEAN_polyvec1_ntt(&s1hat);
PQCLEAN_MLDSA65_CLEAN_polyvec_matrix_pointwise_montgomery(&t1, mat, &s1hat);
PQCLEAN_MLDSA65_CLEAN_polyveck_reduce(&t1);
PQCLEAN_MLDSA65_CLEAN_polyveck_invntt_tomont(&t1);
```

함수 종료 시 mat 할당 해제

```
memset(t->vec[i].coeffs, 0, sizeof(t->vec[i].coeffs));
for(j = 0; j < L; ++j){
    poly_uniform(p, rho, (i << 8) + j);
    poly_pointwise_montgomery(&w, p, &v->vec[i]);
    poly_add(&t->vec[i], &t->vec[i], &w);
```

p 누적(add)

최적 구현

```
int crypto_sign_keypair(uint8_t *pk, uint8_t *sk) {
    uint8_t seedbuf[2*SEEDBYTES + CRHBYTES];
    uint8_t tr[SEEDBYTES];
    const uint8_t *rho, *rhoprime, *key;
    polyvec1 s1, s1hat;
    poly p;
    polyveck s2, t1, t0;
```

함수 시작 시 1KB 할당(이후 값만 교체)

```
/* Get randomness for rho, rhoprime and key */
randombytes(seedbuf, SEEDBYTES);
shake256(seedbuf, 2*SEEDBYTES + CRHBYTES, seedbuf, SEEDBYTES);
rho = seedbuf;
rhoprime = rho + SEEDBYTES;
key = rhoprime + CRHBYTES;
```

```
/* Sample short vectors s1 and s2 */
polyvec1_uniform_eta(&s1, rhoprime, 0);
polyveck_uniform_eta(&s2, rhoprime, L);
```

```
/* Matrix-vector multiplication */
s1hat = s1;
polyvec1_ntt(&s1hat);
```

```
/*
polyvec_matrix_expand(mat, rho);
polyvec_matrix_pointwise_montgomery(&t1, mat, &s1hat);
*/
polyvec_matrix_expand_and_multiply(&t1, rho, &p, &s1hat);
```

p는 루프 내에서 계속
덮어쓰며 재사용

Peak RAM 측정 결과

- Peak RAM 비교
- ML-DSA-65(Dilithium3) 기준
- Valgrind massif 기반

동작	기존 구현	최적 구현	절감률
키젠	68.9KiB	40.9 KiB	-40.6%
서명	87.2KiB	59.3 KiB	-32.0%
검증	65.8KiB	36.7 KiB	-44.2%

성능 측정 결과

- CPU Cycles 비교
- ML-DSA-65(Dilithium3) 기준
- 15,000회 반복 평균
- 서명 중 Rejection sampling 발생 시 재시도
 - 재시도마다 A 재생성 필요 -> 매번 SHAKE128 재호출 필요

동작	기존 구현	최적 구현	증가율
키젠	350 kcc	352 kcc	+0.6%
서명	1,416 kcc	2,081 kcc	+47%
검증	334 kcc	335 kcc	+0.3%

Q & A